

Freight forwarding + hardware shop management system

Technical specification — v1

1. Overview

One web application, one database, two modules, six users.

Module	Purpose	Users
Freight	Order records for goods shipped China → Kigali / Goma / Bukavu	2-3 staff
Shop	Stock ledger and point of sale	2 staff
Shared	Auth, customers, reporting, audit	Manager

The manager is the only account with access to both modules.

2. Roles and permissions

Three roles only. Hardcode them; do not build a permissions editor.

Action	Manager	Freight staff	Shop staff
Create freight order	Yes	Yes	No
Edit order while draft	Yes	Own drafts only	No
Edit order after submit	Yes	No	No
Change order status	Yes	No	No
Record a payment	Yes	Yes	No
Delete an order	Yes	No	No
Record a sale	Yes	No	Yes
Edit or void a sale	Yes	No	No
Manage items and stock	Yes	No	No
Adjust stock	Yes	No	No
View reports	Yes	No	No
Manage users	Yes	No	No

Note: since only the manager changes status, give him a bulk status update screen (tick several orders, set them all to "arrived") and make it work well on a phone. Otherwise this becomes a daily chore.

3. Order lifecycle

draft → submitted → in transit → arrived → delivered
↘ cancelled

- `draft` — visible only to the creating staff member. Fully editable by them.
- `submitted` — appears on manager dashboard. All fields lock. Only manager edits.
- `in transit` → `delivered` — manager only.
- `cancelled` — manager only, requires a reason.

Every transition writes a row to `order_status_history`. Every field change after submit writes a row to `audit_log`.

4. Database schema

Shared

users

Column	Type	Notes
id	bigint PK	
name	varchar	
email	varchar unique	login
password_hash	varchar	
role	enum	manager, freight_staff, shop_staff
is_active	boolean	default true
created_at	timestamp	

customers

Column	Type	Notes
id	bigint PK	
code	varchar unique	e.g. KGL-0412, auto-generated
name	varchar	
phone	varchar	
email	varchar nullable	
id_number	varchar nullable	national ID or passport
address	text nullable	
notes	text nullable	
created_by	FK users	
created_at	timestamp	

audit_log

Column	Type	Notes
id	bigint PK	
table_name	varchar	
record_id	bigint	
field_name	varchar	
old_value	text	
new_value	text	
changed_by	FK users	
changed_at	timestamp	

Freight module

orders

Column	Type	Notes
id	bigint PK	
order_no	varchar unique	e.g. ORD-2026-0087
customer_id	FK customers	
goods_category	enum	vehicle, machinery, electronics, general
description	text	
make	varchar nullable	vehicles
model	varchar nullable	vehicles
year	int nullable	vehicles
colour	varchar nullable	vehicles
vin	varchar nullable unique	chassis number
engine_no	varchar nullable	
quantity	int	default 1
origin_port	varchar	
destination	enum	kigali, goma, bukavu
departure_date	date nullable	leaves China
eta	date nullable	estimated arrival
actual_arrival_date	date nullable	
container_no	varchar nullable	
bl_no	varchar nullable	bill of lading
currency	char(3)	USD, RWF, CDF
total_amount	decimal(14,2)	agreed price
status	enum	see section 3
created_by	FK users	the staff member handling it
submitted_at	timestamp nullable	
cancelled_reason	text nullable	

Column	Type	Notes
created_at / updated_at	timestamp	

Balance is never stored. It is always `total_amount - SUM(order_payments.amount)`.

order_payments

Column	Type	Notes
id	bigint PK	
order_id	FK orders	
amount	decimal(14,2)	
currency	char(3)	
exchange_rate	decimal(12,6)	to base currency, at payment date
method	enum	cash, momo, bank, card
paid_on	date	
proof_image_path	varchar	required
note	varchar nullable	e.g. "deposit", "balance"
recorded_by	FK users	
created_at	timestamp	

order_status_history

Column	Type	Notes
id	bigint PK	
order_id	FK orders	
from_status	varchar	
to_status	varchar	
note	varchar nullable	
changed_by	FK users	
changed_at	timestamp	

Shop module

categories — id, name

suppliers — id, name, phone, email, address

items

Column	Type	Notes
id	bigint PK	
sku	varchar unique	
barcode	varchar nullable	
name	varchar	
category_id	FK categories	
unit	enum	piece, metre, kg, bag, litre, box
cost_price	decimal(12,2)	current weighted average
sell_price	decimal(12,2)	
reorder_level	decimal(12,2)	
is_active	boolean	

stock_movements — the ledger. Never update, only insert.

Column	Type	Notes
id	bigint PK	
item_id	FK items	
movement_type	enum	opening, purchase, sale, return, damage, adjustment
quantity	decimal(12,2)	signed: positive in, negative out
unit_cost	decimal(12,2) nullable	for inbound movements
reference_type	varchar nullable	sale, purchase
reference_id	bigint nullable	
note	varchar nullable	required for adjustment and damage
created_by	FK users	
created_at	timestamp	

Stock on hand = (SELECT SUM(quantity) FROM stock_movements WHERE item_id = ?)

sales

Column	Type	Notes
id	bigint PK	
sale_no	varchar unique	
customer_name	varchar nullable	walk-in, optional
subtotal	decimal(12,2)	
discount	decimal(12,2)	default 0
total	decimal(12,2)	
payment_method	enum	cash, momo, bank, card
proof_image_path	varchar nullable	required unless method = cash
is_voided	boolean	default false
voided_by	FK users nullable	manager only
sold_at	timestamp	
created_by	FK users	

sale_items

Column	Type	Notes
id	bigint PK	
sale_id	FK sales	
item_id	FK items	
quantity	decimal(12,2)	
unit_price	decimal(12,2)	actual price charged
unit_cost_at_sale	decimal(12,2)	snapshot, for profit reporting
line_total	decimal(12,2)	

purchases — id, supplier_id, purchase_no, purchased_at, total, created_by **purchase_items** — id, purchase_id, item_id, quantity, unit_cost

5. Business rules

1. A sale is rejected if it would drive any item's stock below zero.

2. Selling below `sell_price` is permitted but flagged on the manager's dashboard as an under-price sale.
 3. Recording a sale inserts negative `stock_movements` inside the same database transaction as the `sales` row. Never one without the other.
 4. Opening stock is imported once as `movement_type = opening`, then the import screen is disabled.
 5. Voiding a sale does not delete it. It sets `is_voided`, and inserts reversing positive stock movements.
 6. `cost_price` is recalculated as a weighted average on every purchase.
 7. Proof of payment images are compressed to under 500 KB on upload; store the file path in the database, never the binary.
 8. An order cannot move to `delivered` while a balance is outstanding, unless the manager explicitly overrides with a note.
-

6. Screens

Freight staff

1. Login
2. My orders — list, filter by status
3. New order — the form (customer, goods, shipping, payment)
4. Order detail — read-only once submitted, with a button to add a payment
5. Customers — list, add new

Shop staff

1. Login
2. Sale screen — item search, cart, payment method, proof upload, complete
3. Today's sales — own sales only
4. Stock lookup — read-only quantity check

Manager

1. Dashboard — orders awaiting review, arrivals in next 14 days, total outstanding balances, low stock alerts, today's shop sales, under-price flags
2. All orders — filter by staff, status, destination, date
3. Order detail — full edit, status change, payment history, audit trail
4. Bulk status update
5. Customers
6. Items — master list, add, edit, deactivate
7. Opening stock import
8. Purchases — record goods received
9. Stock movements — full ledger, filterable
10. Stock adjustments — with mandatory reason

11. Low stock alerts
 12. Reports (section 7)
 13. Users — add, deactivate, reset password
 14. Audit log
-

7. Reports

Freight

- Orders per month — count and value
- Outstanding balances by customer
- Orders per staff member
- Average transit time, departure to actual arrival, by destination
- Top customers by value

Shop

- Daily and monthly sales
- Profit per item — $(\text{unit_price} - \text{unit_cost_at_sale}) \times \text{quantity}$
- Stock on hand and total stock value
- Slow movers — no sale in 60 days
- Sales per staff member
- Under-price sales

Combined

- Monthly summary: revenue per module, total cash in, outstanding receivables

Every report exportable to Excel and PDF.

8. Deployment

- Web application on a small cloud VPS: 2 vCPU, 4 GB RAM, 50 GB disk
 - Shop sale screen built as a PWA with a local queue, so the counter keeps working during internet or power outages
 - UPS on the counter machine and the router
 - Automated daily database backup to off-server storage, retained 30 days
 - Uploaded images backed up separately from the database
-

9. Build order

Phase	Deliverable
1	Auth, users, roles, customers, audit log
2	Items, opening stock import, stock movements ledger
3	Sale screen, stock deduction, low stock alerts
4	Freight order form, draft/submit lock, payments
5	Manager dashboard, status updates, order editing
6	Reports and exports
7	PWA offline mode, backups, hardening

Phases 2 and 3 give a working, useful system on their own. Ship them before starting phase 4.

10. Open items

- RRA / EBM 2.1 registration and integration for shop receipts and VAT. Start the registration process early; it takes longer than the coding.
- Decide the base reporting currency (recommended: USD for freight, RWF for the shop, with the combined report converted at the rate stored per payment).
- Confirm whether Goma and Bukavu deliveries need separate customs document storage.